UNIVERSITÉ HASSAN II DE CASABLANCA

FACULTÉ DES SCIENCES BEN M'SICK

DÉPARTEMENT DE MATHÉMATIQUES ET INFORMATIQUE

Master d'excellence en intelligence artificielle

RAPPORT DE PROJET DE FIN DE MODULE :

Le Cloud Computing & DevOps

Mise en place d'une infrastructure DevOps avec CI/CD et déploiement sur Kubernetes

Réalisé par :

Hajar En-najdy
Mohammed Akrame Kraichy
Ayoub Hmiad

Exposé le 24 Mars 2026 devant les membres de jury :

Pr. F. Benabbou	Faculté des sciences Ben M'Sick	Encadrante
Pr. Abderahmane DAIF	Faculté des sciences Ben M'Sick	Encadrant

Année Universitaire : 2025 - 2026

Dédicace

Nous dédions ce travail à nos familles, qui ont été une source inépuisable de soutien, d'encouragement et de patience tout au long de notre parcours.
Leur confiance et leur présence constante ont nourri notre motivation et renforcé notre détermination à mener ce projet à bien.

Nous dédions également ce rapport à nos enseignants, dont l'engagement, la bienveillance et la rigueur ont contribué à forger notre sens de l'effort et notre exigence académique.

Enfin, nous pensons à toutes les personnes qui, de près ou de loin, ont marqué notre chemin et nous ont accompagnés dans cette aventure.
Ce travail est aussi le fruit de leur soutien.

HAJAR EN-NAJDY & MOHAMED AKRAME KRAICHY & AYOUB HMIAD

Remerciements

Nous tenons à exprimer notre plus sincère gratitude à toutes les personnes qui ont rendu possible la réalisation de ce travail collectif.

Tout d'abord, un merci tout particulier à **Mme FOUZIA BENABBOU**, enseignante du module Cloud Computing & DevOps. Ses cours clairs, pratiques et passionnés nous ont transmis les fondations techniques solides sans lesquelles ce projet n'aurait pu voir le jour.

Nos remerciements vont également à notre encadrant **Pr. Abderahmane DAIF** pour son écoute attentive, ses remarques toujours pertinentes, son soutien technique précieux et sa bienveillance qui nous ont permis de dépasser de nombreux obstacles.

Nous remercions également l'ensemble des enseignants et encadrants du master pour leurs connaissances partagées, leur enthousiasme et la qualité de la formation qui nous a menés jusqu'ici.

Enfin, une pensée émue pour nos familles et nos proches respectifs, qui ont supporté nos emplois du temps chargés, nos silences prolongés et nos moments de stress, tout en nous offrant un soutien moral et affectif indispensable.

Table des matières

Dédicace 2
Remerciements 3
1. Introduction 6
2. Architecture globale du projet 7
2.1 Gestion du code source – GitHub 7
2.2 Pipeline CI/CD – GitHub Actions 7
2.3 Conteneurisation – Docker 7
2.4 Infrastructure as Code – Terraform 7
2.5 Configuration automatisée – Ansible 7
2.6 Orchestration – Kubernetes (k3s) 8
2.7 Monitoring – Prometheus & Grafana 8
2.8 Flux global de fonctionnement 8
3. Développement de l'application 10
3.1 Implémentation basée sur Express 10
3.2 Structure du projet 12
4. Conteneurisation avec Docker 13
5. Infrastructure avec Terraform 15
6. Configuration avec Ansible 17
7. Déploiement avec Kubernetes 19
8. Pipeline CI/CD 21
8.1 Fonctionnement du pipeline 21
8.1.1 Phase de test 21
8.1.2 Phase de build 21
8.1.3 Phase de déploiement 21
8.2 Apports du pipeline CI/CD 22
9. Tests automatisés 24
10. Monitoring 25
11. Résultats 27
12. Conclusion 28

Table de figures

Figure 1 : Architecture globale du système 8
Figure 2 : Implémentation de l'API REST avec Express 11
Figure 3 : Exemple d'exécution et test de l'API 11
Figure 4 : Structure globale du projet DevOps 12
Figure 5 : Construction de l'image Docker à partir du Dockerfile 13
Figure 6 : Fichier Dockerfile de l'application 14
Figure 7 : Exécution du conteneur Docker et démarrage de l'application 14
Figure 8 : Extrait du fichier Terraform (main.tf) pour le provisionnement de l'infrastructure 15
Figure 9 : Exécution de Terraform et création de l'infrastructure (machines master et worker) 16
Figure 10 : Ressources cloud créées sur Azure via Terraform 16
Figure 11 : Extrait du playbook Ansible pour la configuration des machines 17
Figure 12 : Résultat de l'exécution du playbook Ansible (PLAY RECAP) 18
Figure 13 : Définition du Deployment Kubernetes (deployment.yaml) 19
Figure 14 : Définition du service Kubernetes (service.yaml) de type NodePort 20
Figure 15 : Fichier de configuration du pipeline CI/CD 22
Figure 16 : Exécution réussie du pipeline CI/CD avec GitHub Actions 23
Figure 17 : État des cibles surveillées dans Prometheus 25
Figure 18 : Tableau de bord Grafana illustrant les métriques du système 26
Figure 19 : Vérification de l'état de l'application via endpoint /api/health 27

1. Introduction

Le développement logiciel contemporain est marqué par une exigence croissante de rapidité de mise sur le marché, tout en garantissant des standards élevés de qualité, de sécurité et de fiabilité. Dans ce contexte, les approches traditionnelles, caractérisées par une séparation entre les équipes de développement (Dev) et d'exploitation (Ops), montrent rapidement leurs limites.

C'est pour répondre à ces contraintes que le paradigme **DevOps** s'est imposé, en proposant une intégration continue des processus de développement, de test et de déploiement, appuyée par l'automatisation et une forte collaboration entre les équipes. Cette approche permet de réduire les délais de livraison, d'améliorer la qualité des applications et de faciliter leur mise en production.

Dans ce cadre, le présent projet vise à concevoir et mettre en œuvre une infrastructure DevOps complète, reproduisant les pratiques utilisées dans les environnements cloud modernes. Il s'articule autour de plusieurs axes principaux :

- le développement d'une application web basée sur une API REST ;
- sa conteneurisation à l'aide de Docker afin d'assurer sa portabilité ;
- la mise en place d'un pipeline CI/CD automatisé pour gérer les phases de test, de build et de déploiement ;
- le déploiement et l'orchestration de l'application dans un cluster Kubernetes ;
- l'intégration d'un système de monitoring pour assurer la supervision de l'infrastructure.

L'ensemble de ces composants permet de simuler une chaîne DevOps complète, depuis le développement jusqu'à la mise en production, en s'appuyant sur des outils largement utilisés dans l'industrie tels que Docker, Kubernetes, GitHub Actions, Terraform, Ansible et Grafana.

Ce projet constitue ainsi une mise en pratique concrète des concepts fondamentaux du DevOps, tout en mettant en évidence les enjeux liés à l'automatisation, à la scalabilité et à la gestion d'infrastructures distribuées.

2. Architecture globale du projet

L'architecture du projet repose sur une approche modulaire intégrant plusieurs outils DevOps afin d'automatiser l'ensemble du cycle de vie de l'application, depuis le développement jusqu'au déploiement et au monitoring. Elle s'inscrit dans le paradigme **DevOps et cloud-native**, en combinant des solutions d'intégration continue, de conteneurisation, d'orchestration et de supervision.

2.1 Gestion du code source – GitHub

Le code source de l'application est hébergé sur la plateforme GitHub, qui constitue le socle de gestion de version et de collaboration. Elle permet de suivre les évolutions du projet, de faciliter le travail en équipe et de déclencher automatiquement le pipeline CI/CD lors des modifications du code, notamment lors des opérations de push ou de pull request.

2.2 Pipeline CI/CD – GitHub Actions

Le pipeline CI/CD, implémenté à l'aide de GitHub Actions, automatise les différentes étapes du cycle de vie logiciel. À chaque modification du code, les tests sont exécutés automatiquement, une image Docker est construite, puis l'application est déployée sur le cluster Kubernetes. Cette automatisation permet d'assurer une livraison continue, rapide et fiable, tout en réduisant les erreurs humaines.

2.3 Conteneurisation – Docker

L'application est conteneurisée à l'aide de Docker afin de garantir sa portabilité et la cohérence de son exécution. En encapsulant l'ensemble des dépendances dans une image standardisée, Docker permet de simplifier le déploiement et d'assurer un fonctionnement identique sur tous les environnements.

2.4 Infrastructure as Code – Terraform

Terraform est utilisé pour provisionner l'infrastructure cloud selon le principe de l'Infrastructure as Code. Il permet de créer automatiquement les machines virtuelles nécessaires, notamment un nœud master et un nœud worker, tout en offrant la possibilité de reproduire facilement l'environnement à partir des fichiers de configuration.

2.5 Configuration automatisée – Ansible

Ansible intervient après le provisionnement pour configurer les machines. Il permet d'installer Docker, de déployer Kubernetes (k3s) et de configurer le cluster. Grâce à cette automatisation, le nœud worker peut rejoindre le nœud master de manière fiable et reproductible.

2.6 Orchestration – Kubernetes (k3s)

Kubernetes assure l'orchestration des conteneurs en prenant en charge le déploiement et la gestion de l'application. Il permet de gérer les pods, d'exposer les services et d'assurer la tolérance aux pannes grâce à des mécanismes d'auto-récupération.

2.7 Monitoring – Prometheus & Grafana

Un système de monitoring est mis en place afin de superviser l'infrastructure. Prometheus collecte les métriques en continu, tandis que Grafana permet de les visualiser sous forme de tableaux de bord. Cette combinaison offre une meilleure visibilité sur le système et facilite la détection des anomalies.

2.8 Flux global de fonctionnement

Le fonctionnement global de l'architecture est le suivant :

1. Le développeur pousse le code sur GitHub
2. Le pipeline CI/CD est déclenché automatiquement
3. Les tests sont exécutés
4. L'image Docker est construite et publiée
5. Kubernetes déploie l'application
6. Prometheus collecte les métriques
7. Grafana affiche les performances

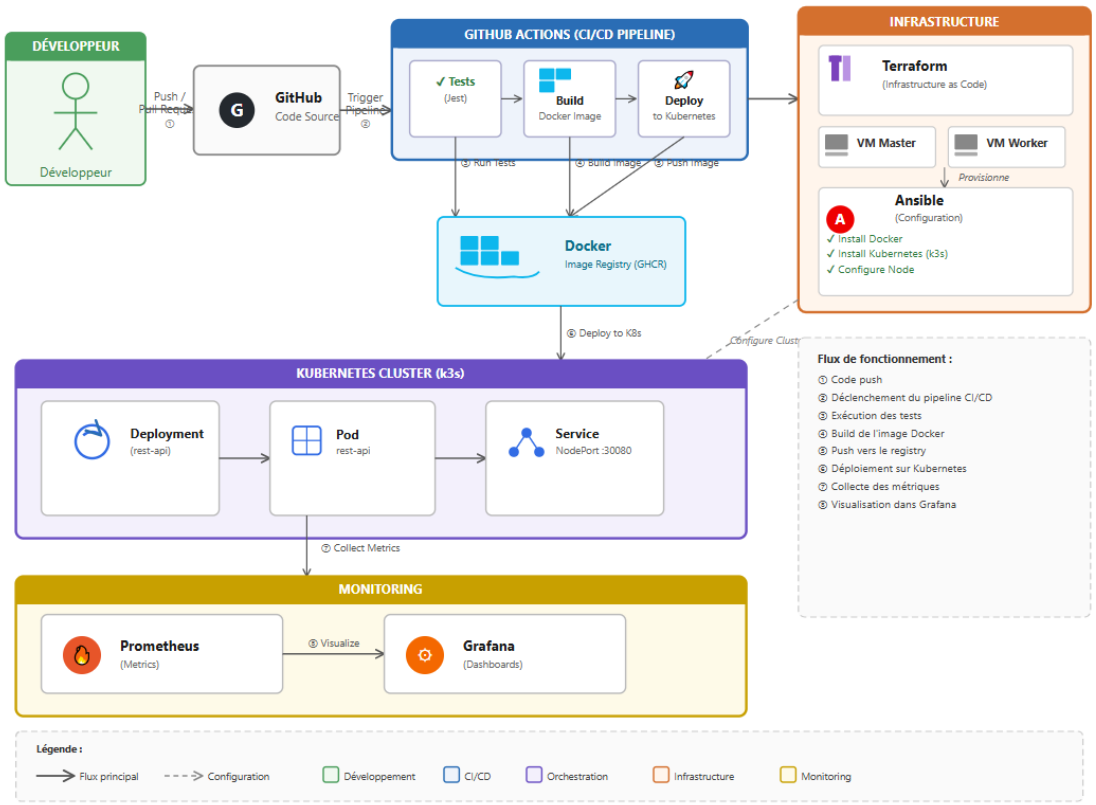

Figure 1 : Architecture globale du système

Cette architecture nous a permis de comprendre concrètement l'intégration des outils DevOps dans une chaîne automatisée complète, proche des environnements professionnels.

3. Développement de l’application

Dans le cadre de ce projet, une application web a été développée sous forme d’une **API REST** en utilisant **Node.js et le framework Express**. Ce choix s’explique par la simplicité, la légèreté et l’efficacité d’Express pour la mise en place rapide de services backend.

L’application permet de gérer une liste d’éléments à travers les opérations classiques de type **CRUD** (*Create, Read, Update, Delete*). Ce type d’application a été retenu afin de disposer d’un cas d’usage simple mais suffisamment représentatif pour être intégré dans une chaîne DevOps complète.

Sur le plan fonctionnel, l’API offre la possibilité d’ajouter, de modifier et de supprimer des éléments, ainsi que de consulter leur état. Elle permet également d’accéder à des statistiques globales, telles que le nombre total d’éléments ou leur état d’avancement.

L’architecture de l’application repose sur un serveur Express structuré autour de routes REST bien définies. Les données sont stockées temporairement en mémoire, ce qui permet de simplifier l’implémentation tout en conservant un comportement réaliste pour les besoins du projet.

3.1 Implémentation basée sur Express

L’API expose plusieurs endpoints principaux. Une route permet d’obtenir des informations générales sur le service, tandis que d’autres routes assurent la gestion complète des éléments (création, modification, suppression et consultation). Une route spécifique est également dédiée à la vérification de l’état du serveur, ce qui permet notamment son intégration avec les mécanismes de supervision et de contrôle de Kubernetes.

Afin d’assurer la robustesse de l’application, un mécanisme de gestion des erreurs a été mis en place. Celui-ci permet de capturer les erreurs éventuelles et de retourner des réponses claires et structurées au client, garantissant ainsi une meilleure stabilité du service.

Par ailleurs, des tests automatisés ont été développés à l’aide de l’outil **Jest**, permettant de valider le bon fonctionnement des différentes fonctionnalités de l’API. Ces tests sont intégrés dans le pipeline CI/CD afin de vérifier automatiquement la qualité du code avant chaque déploiement.

```js
const express = require('express');
const cors = require('cors');
const path = require('path');
const app = express();
const PORT = process.env.PORT || 3000;

// ==================== MIDDLEWARE ====================
app.use(express.json());
app.use(express.static('public'));
app.use(cors());

// Request logging middleware
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.path}`);
  next();
});

// ==================== ERROR HANDLING MIDDLEWARE ====================
const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};

// ==================== DATA STORAGE (In-memory for demo) ====================
let items = [
  { id: 1, title: 'Learn Node.js', description: 'Master Express and REST APIs', completed: false, createdAt: new Date('2024-01-15') },
  { id: 2, title: 'Build a REST API', description: 'Create a fully functional API with CRUD operations', completed: true, createdAt: new Date('2024-01-20') },
  { id: 3, title: 'Add database', description: 'Integrate MongoDB or PostgreSQL', completed: false, createdAt: new Date('2024-02-01') }
];

let nextId = 4;

// ==================== ROUTES ====================

// GET - Main API info
app.get('/api', (req, res) => {
  res.json({
    message: 'Welcome to the REST API',
    version: '2.0.0',
    status: 'running',
```

Figure 2 : Implémentation de l'API REST avec Express

Cette figure présente un extrait du code de l'application développée avec Node.js et Express. Elle illustre la configuration du serveur, la gestion des middlewares ainsi que l'implémentation des routes de l'API.

Enfin, l'application a été conçue de manière à s'intégrer facilement dans une démarche DevOps. Elle est conteneurisable, testable automatiquement et déployable dans un environnement Kubernetes, ce qui permet de reproduire un cycle de vie logiciel proche des conditions réelles de production.

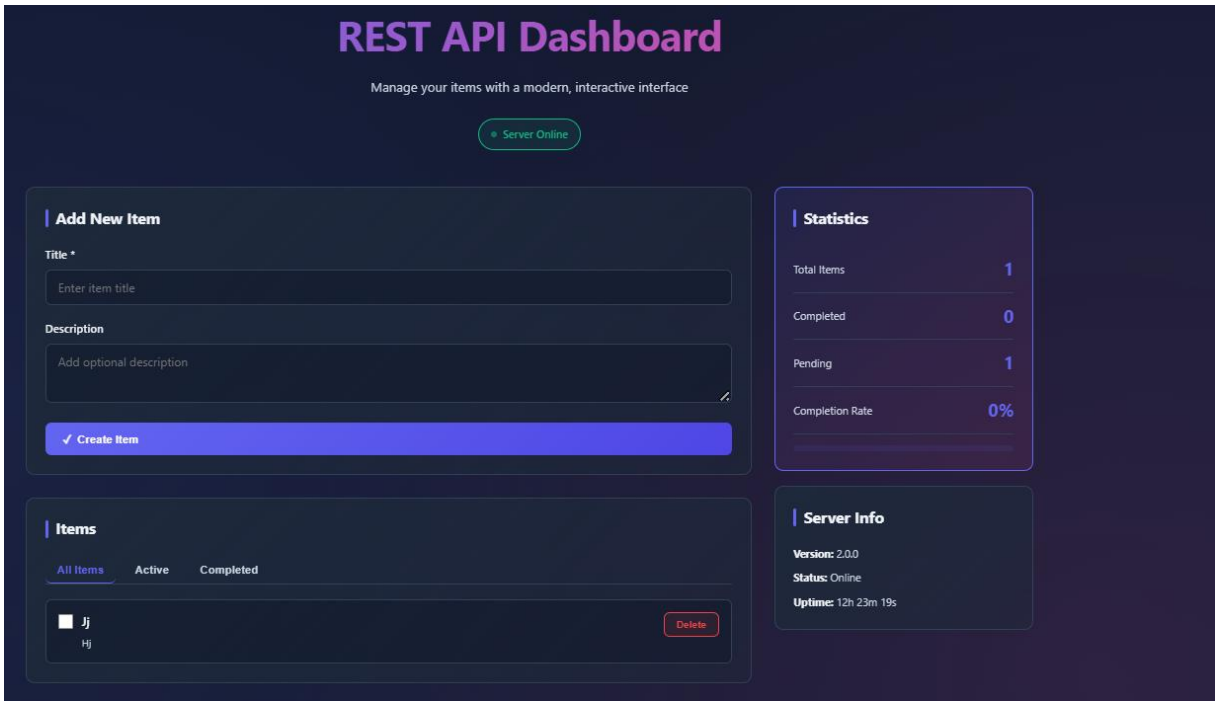

Figure 3 : Exemple d'exécution et test de l'API

Cette interface illustre le bon fonctionnement de l'application après son déploiement. Elle permet d'interagir avec l'API en temps réel pour créer, supprimer et consulter des éléments.

Les données affichées, notamment les statistiques et l'état du serveur, sont récupérées dynamiquement depuis le backend, confirmant la bonne communication entre le client et le serveur ainsi que la disponibilité de l'API.

3.2 Structure du projet

Afin d'assurer une organisation claire et modulaire du projet, une structure bien définie a été adoptée. Celle ci permet de séparer les différentes composantes de l'application ainsi que les outils DevOps utilisés.

```
devops-app-main/
├── .github/
│   └── workflows/
│       └── ci-cd.yml
├── ansible/
│   ├── ansible.cfg
│   ├── inventory.ini.example
│   └── playbook.yml
├── app/
│   ├── Dockerfile
│   ├── package.json
│   ├── server.js
│   ├── public/
│   │   └── index.html
│   └── tests/
│       └── app.test.js
├── k8s/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── monitoring.yaml
├── terraform/
│   ├── main.tf
│   └── outputs.tf
└── .gitignore
```

Figure 4 : Structure globale du projet DevOps

Le projet est organisé comme suit :

- app/ : contient le code source de l'application (backend, interface, tests)
- ansible/ : contient les fichiers de configuration pour l'automatisation des machines
- terraform/ : permet le provisionnement de l'infrastructure cloud
- k8s/ : regroupe les fichiers de déploiement Kubernetes (Deployment, Service, Monitoring)
- .github/workflows/ : contient le pipeline CI/CD (GitHub Actions)

Cette organisation facilite la maintenance, la compréhension du projet et son évolution.

4. Conteneurisation avec Docker

Dans le cadre de ce projet, l'application a été conteneurisée à l'aide de **Docker** afin de garantir sa portabilité et de faciliter son déploiement dans différents environnements.

La conteneurisation consiste à encapsuler l'application ainsi que toutes ses dépendances (bibliothèques, runtime, configurations) dans une image unique et reproductible. Cette approche permet d'assurer que l'application fonctionne de manière identique, indépendamment de l'environnement d'exécution (machine locale, serveur cloud ou cluster Kubernetes).

L'utilisation de Docker répond principalement à deux objectifs. D'une part, elle permet d'éliminer les problèmes liés aux différences d'environnement entre développement et production. D'autre part, elle simplifie considérablement le processus de déploiement en fournissant une unité standardisée facilement transportable.

La mise en œuvre de cette conteneurisation s'est déroulée en plusieurs étapes. Tout d'abord, un fichier **Dockerfile** a été créé afin de définir les différentes instructions nécessaires à la construction de l'image, telles que l'installation des dépendances Node.js et la configuration du serveur. Ensuite, l'image Docker a été construite à partir de ce fichier. Enfin, un conteneur a été lancé à partir de cette image afin de vérifier le bon fonctionnement de l'application dans un environnement isolé.

Cette étape constitue un élément central de l'architecture DevOps, puisqu'elle permet d'intégrer l'application dans le pipeline CI/CD et de la déployer facilement au sein du cluster Kubernetes.

```
C:\Users\FUJITSU\Downloads\files>docker build -t monapp .
[+] Building 16.9s (11/11) FINISHED                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                   0.1s
 => => transferring dockerfile: 433B                                                                   0.0s
 => [internal] load metadata for docker.io/library/node:18-alpine                                      2.3s
 => [auth] library/node:pull token for registry-1.docker.io                                            0.0s
 => [internal] load .dockerignore                                                                      0.0s
 => => transferring context: 2B                                                                        0.0s
 => [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8c  9.3s
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8c  0.0s
 => => sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 446B / 446B             0.4s
 => => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 1.26MB / 1.26MB         0.9s
 => => sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e 40.01MB / 40.01MB       6.8s
 => => sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB         2.3s
 => => extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870              0.3s
 => => extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4efea161b2b5a3d0e              2.2s
 => => extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3              0.1s
 => => extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549              0.0s
 => [internal] load build context                                                                      0.5s
 => => transferring context: 171.76kB                                                                  0.4s
 => [2/5] WORKDIR /app                                                                                 0.2s
 => [3/5] COPY package*.json ./                                                                        0.1s
 => [4/5] RUN npm install --omit=dev                                                                   3.2s
 => [5/5] COPY app.js .                                                                                0.1s
 => exporting to image                                                                                 1.3s
```

Figure 5 : Construction de l'image Docker à partir du Dockerfile

Cette figure illustre le processus de construction de l'image Docker à partir du fichier Dockerfile. Les différentes étapes affichées correspondent à l'installation des dépendances, à la configuration de l'environnement d'exécution et à la préparation de l'application.

```
! service.yaml    Release Notes: 1.112.0    ! deployment.yaml    Dockerfile ×
C: > Users > HP > Downloads > devops-app-main > devops-app-main > app > Dockerfile > FROM
1  FROM node:18-alpine
2  WORKDIR /app
3  COPY package*.json ./
4  RUN npm install
5  COPY . .
6  EXPOSE 3000
7  ENV NODE_ENV=production
8  CMD ["node", "server.js"]
9
```

Figure 6 : Fichier Dockerfile de l'application

Cette figure présente le fichier Dockerfile utilisé pour construire l'image de l'application.
Il définit l'environnement d'exécution basé sur Node.js, installe les dépendances nécessaires, copie le code source et configure la commande de lancement du serveur.

La réussite de cette opération confirme que l'image a été correctement générée et qu'elle est prête à être utilisée pour le déploiement.

```
 => => unpacking to docker.io/library/monapp:latest

C:\Users\FUJITSU\Downloads\files>docker run -p 3000:3000 monapp
Serveur démarré sur le port 3000
```

Figure 7 : Exécution du conteneur Docker et démarrage de l'application

Cette figure montre l'exécution du conteneur Docker à partir de l'image précédemment construite. La commande utilisée permet d'exposer l'application sur le port 3000, rendant ainsi le service accessible.
Le message indiquant le démarrage du serveur confirme que l'application fonctionne correctement dans un environnement conteneurisé.

La conteneurisation constitue ainsi une étape clé dans la standardisation des déploiements et dans l'industrialisation du cycle de vie applicatif.

5. Infrastructure avec Terraform

Dans le cadre de ce projet, **Terraform** a été utilisé afin d'automatiser le provisionnement de l'infrastructure cloud selon le principe de l'**Infrastructure as Code (IaC)**.

Cette approche consiste à définir l'ensemble des ressources nécessaires (machines virtuelles, réseau, configurations) sous forme de code déclaratif, ce qui permet de créer et de gérer l'infrastructure de manière automatisée, reproductible et versionnable.

Concrètement, Terraform a été utilisé pour déployer deux machines virtuelles sur le cloud :

- une machine jouant le rôle de **nœud master**, destinée à héberger le plan de contrôle Kubernetes ;
- une machine configurée en tant que **nœud worker**, chargée d'exécuter les applications conteneurisées.

En parallèle, les règles d'accès réseau ont été définies afin de permettre la communication entre les différentes machines ainsi que l'accès distant (notamment via SSH), garantissant ainsi le bon fonctionnement du cluster.

L'utilisation de Terraform présente plusieurs avantages majeurs. Elle permet tout d'abord de **reproduire facilement l'infrastructure** sur différents environnements, simplement en réexécutant les scripts. Elle offre également un **gain de temps considérable**, en automatisant des tâches qui seraient autrement longues et sujettes aux erreurs lorsqu'elles sont réalisées manuellement.

main.tf

C: > Users > HP > Downloads > devops-app-main > devops-app-main > terraform > main.tf

```
10  provider "azurerm" {
13  }
14
15  # Variables
16
17  variable "location" {
18    default = "West Europe"
19  }
20
21  variable "admin_username" {
22    default = "azureuser"
23  }
24
25  variable "ssh_public_key_path" {
26    default = "~/.ssh/id_rsa.pub"
27  }
28
29  variable "vm_size" {
30    default = "Standard_B2s"
31  }
32
33  #  Resource Group
34
35  resource "azurerm_resource_group" "rg" {
36    name     = "rg-devops-project"
37    location = var.location
38  }
```

Figure 8 : Extrait du fichier Terraform (main.tf) pour le provisionnement de l'infrastructure

Cette figure illustre un extrait du fichier de configuration Terraform (main.tf) utilisé pour le provisionnement de l'infrastructure cloud.
On y retrouve la définition des variables principales telles que la localisation, le type de machine virtuelle et les paramètres d'accès, ainsi que la création du groupe de ressources.

Cette étape constitue une base essentielle du projet, puisqu'elle prépare l'environnement nécessaire à l'installation de Kubernetes et au déploiement de l'application.

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

master_private_ip = "10.0.1.5"
master_public_ip = "40.115.36.186"
worker_private_ip = "10.0.1.4"
worker_public_ip = "20.73.81.145"
```

Figure 9 : Exécution de Terraform et création de l'infrastructure (machines master et worker)

Cette figure montre le résultat de l'exécution de la commande terraform apply. Le message de confirmation ainsi que les adresses IP générées indiquent que les machines virtuelles ont été correctement créées et sont prêtes à être utilisées dans le cluster Kubernetes.

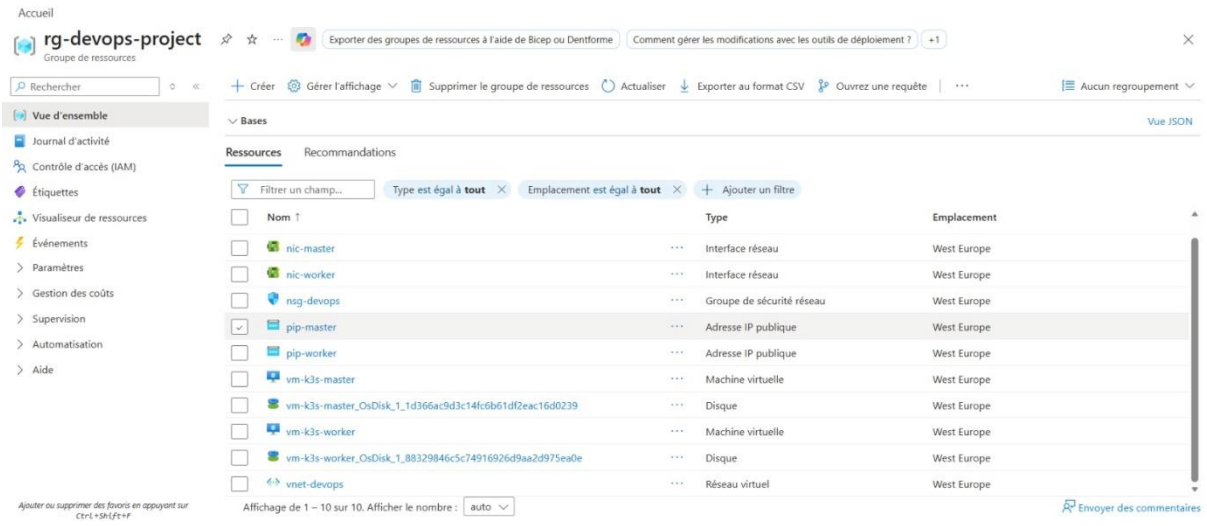

Figure 10 : Ressources cloud créées sur Azure via Terraform

Cette figure présente les ressources cloud créées automatiquement à l'aide de Terraform. On y retrouve les machines virtuelles, les interfaces réseau et les adresses IP, confirmant le bon provisionnement de l'infrastructure.

6. Configuration avec Ansible

Dans ce projet, **Ansible** a été utilisé pour automatiser la configuration des machines provisionnées précédemment avec Terraform. Cette étape s'inscrit dans la logique de l'**Infrastructure as Code**, en permettant de définir et d'appliquer automatiquement l'ensemble des configurations nécessaires au bon fonctionnement du système.

Ansible fonctionne selon une approche **agentless**, en utilisant le protocole SSH pour exécuter des tâches à distance sur les machines cibles, ce qui simplifie considérablement son déploiement et son utilisation.

La configuration a été réalisée à l'aide d'un **playbook**, regroupant une série de tâches exécutées de manière structurée sur les différents nœuds du cluster.

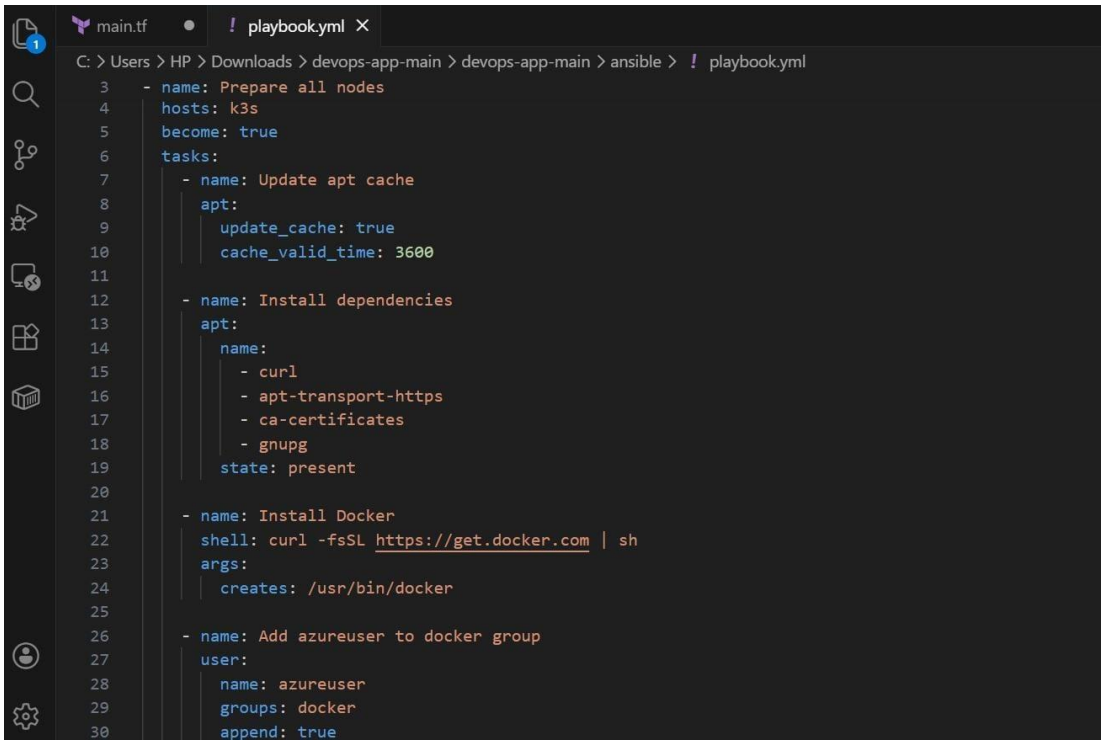

Figure 11 : Extrait du playbook Ansible pour la configuration des machines

Le playbook contient une suite ordonnée de tâches destinées à configurer ou déployer un parc de machines.
Le playbook décrit l'état souhaité de l'infrastructure, et Ansible s'occupe de mettre en œuvre les changements nécessaires pour atteindre cet état.

Dans un premier temps, les machines ont été préparées en installant les dépendances nécessaires, notamment Docker, indispensable à l'exécution des conteneurs. Ensuite, Kubernetes a été installé sous sa forme légère **k3s**, adaptée aux environnements de test et aux infrastructures simplifiées.

Le nœud **master** a été configuré pour initialiser le cluster Kubernetes et générer un token d'authentification. Ce token a ensuite été utilisé pour permettre au nœud **worker** de rejoindre

automatiquement le cluster, assurant ainsi une communication correcte entre les différentes machines.

Grâce à cette automatisation, l'ensemble du processus de configuration est reproductible, rapide et fiable, réduisant fortement les erreurs humaines liées à une configuration manuelle.

Au terme de cette étape :

- le nœud master est entièrement opérationnel ;
- le nœud worker est correctement intégré au cluster Kubernetes ;
- l'infrastructure est prête pour le déploiement de l'application.

Cette phase constitue un élément clé du projet, car elle assure la transition entre le provisionnement de l'infrastructure et l'orchestration des applications.

```
ok: [vm-master]

TASK [Apply manifests] ************************************************************************************************************
changed: [vm-master]

PLAY RECAP ************************************************************************************************************************
vm-master                  : ok=13   changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
vm-worker                  : ok=8    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Figure 12 : Résultat de l'exécution du playbook Ansible (PLAY RECAP)

Cette figure présente le résumé de l'exécution du playbook Ansible. Les résultats indiquent que les tâches ont été exécutées avec succès sur les nœuds master et worker, sans aucune erreur, confirmant la bonne configuration des machines.

7. Déploiement avec Kubernetes

Après la conteneurisation de l'application et la mise en place de l'infrastructure, l'étape suivante a consisté à déployer l'application sur un cluster **Kubernetes**. Pour cela, la distribution légère **k3s** a été utilisée, adaptée aux environnements de test et aux infrastructures simplifiées.

Kubernetes joue ici un rôle central en assurant l'**orchestration des conteneurs**, c'est-à-dire leur déploiement, leur gestion et leur supervision de manière automatisée.

Le déploiement de l'application repose sur deux ressources principales définies à l'aide de fichiers YAML :

- un **Deployment**, responsable de la création et de la gestion des pods ;
- un **Service de type NodePort**, permettant d'exposer l'application vers l'extérieur.

Le **Deployment** permet de définir l'état souhaité de l'application. Il garantit que le nombre de pods spécifié est toujours respecté et assure automatiquement leur redémarrage en cas de défaillance. Il intègre également des mécanismes de vérification tels que les *liveness* et *readiness probes*, qui permettent de surveiller l'état de santé de l'application.

```
! service.yaml    Release Notes: 1.112.0    ! deployment.yaml ×
C: > Users > HP > AppData > Local > Temp > Rar$DIa13124.21452 > ! deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: rest-api
5    labels:
6      app: rest-api
7  spec:
8    replicas: 1
9    selector:
10     matchLabels:
11       app: rest-api
12   template:
13     metadata:
14       labels:
15         app: rest-api
16     spec:
17       containers:
18         - name: rest-api
19           image: ghcr.io/nanigashi011/flask-devops-app:latest
20           ports:
21             - containerPort: 3000
22           livenessProbe:
23             httpGet:
24               path: /api/health
25               port: 3000
26             initialDelaySeconds: 10
27             periodSeconds: 10
28           readinessProbe:
```

Figure 13 : Définition du Deployment Kubernetes (deployment.yaml)

Cette figure présente le fichier de configuration Kubernetes (deployment.yaml) utilisé pour déployer l'application.
Le Deployment définit le nombre de réplicas, l'image Docker utilisée ainsi que les paramètres de surveillance tels que les probes de santé (liveness et readiness).

Il permet d'assurer le déploiement, la gestion et la disponibilité continue de l'application dans le cluster.

Le **Service NodePort** permet quant à lui de rendre l'application accessible depuis l'extérieur du cluster en exposant un port spécifique. Cela permet d'accéder à l'API via l'adresse IP du nœud et le port configuré.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: rest-api-service
spec:
  type: NodePort
  selector:
    app: rest-api
  ports:
    - protocol: TCP
      port: 80
      targetPort: 3000
      nodePort: 30080
```

Figure 14 : Définition du service Kubernetes (service.yaml) de type NodePort

Cette figure présente le fichier de configuration Kubernetes (service.yaml) utilisé pour exposer l'application.
Le service est de type NodePort, ce qui permet d'accéder à l'application depuis l'extérieur du cluster via un port spécifique (30080).
Il assure la redirection du trafic vers le port interne du conteneur (3000), garantissant ainsi l'accessibilité de l'application.

Le fonctionnement global est le suivant : Kubernetes déploie les pods contenant l'application, assure leur disponibilité et redirige le trafic entrant vers ces pods via le service. En cas de panne d'un pod, Kubernetes le recrée automatiquement, garantissant ainsi la continuité du service.

Cette étape permet de démontrer la capacité de Kubernetes à gérer efficacement des applications conteneurisées dans un environnement distribué, tout en assurant leur scalabilité et leur résilience.

8. Pipeline CI/CD

Dans le cadre de ce projet, un pipeline **CI/CD (Continuous Integration / Continuous Deployment)** a été mis en place à l'aide de **GitHub Actions**, afin d'automatiser l'ensemble du cycle de vie de l'application, depuis la validation du code jusqu'à son déploiement sur Kubernetes.

Le pipeline est déclenché automatiquement à chaque modification du code source (notamment lors d'un *push* sur la branche principale), ce qui permet d'intégrer rapidement les changements tout en garantissant leur qualité.

8.1 Fonctionnement du pipeline

Le pipeline est structuré en trois étapes principales :

8.1.1 Phase de test

Lors de cette première étape, les dépendances de l'application sont installées, puis les tests automatisés sont exécutés.

Cette phase permet de vérifier que les nouvelles modifications n'introduisent pas d'erreurs et que l'application reste fonctionnelle. En cas d'échec des tests, le pipeline est interrompu, empêchant ainsi le déploiement d'une version instable.

8.1.2 Phase de build

Une fois les tests validés, une image Docker de l'application est construite automatiquement. Cette image est ensuite publiée dans un registre de conteneurs, afin d'être utilisée pour le déploiement.

Cette étape garantit que l'application est prête à être exécutée dans un environnement isolé et standardisé.

8.1.3 Phase de déploiement

Dans la dernière étape, l'image Docker nouvellement créée est déployée automatiquement sur le cluster Kubernetes.

Le pipeline se connecte au serveur distant via SSH, applique les fichiers de configuration Kubernetes et met à jour l'image utilisée par le déploiement. Kubernetes se charge ensuite de redéployer l'application avec la nouvelle version.

```yaml
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

env:
  IMAGE_NAME: flask-devops-app
  REGISTRY: ghcr.io

jobs:
  test:
    name: Run Tests
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "18"

      - name: Install dependencies
        run: npm install
        working-directory: app
```

Figure 15 : Fichier de configuration du pipeline CI/CD

Cette figure présente le fichier de configuration du pipeline CI/CD (ci-cd.yml) utilisé avec GitHub Actions.
Il définit les différentes étapes du workflow, notamment l'exécution des tests, l'installation des dépendances, la construction de l'image Docker et le déploiement de l'application. Ce pipeline est déclenché automatiquement à chaque modification du code, garantissant une intégration et un déploiement continus.

8.2 Apports du pipeline CI/CD

La mise en place de ce pipeline présente plusieurs avantages majeurs :

- une **automatisation complète** du processus de déploiement
- une **réduction des erreurs humaines** grâce à la standardisation des étapes
- un **gain de temps considérable** lors des mises à jour
- une **amélioration de la qualité du code** grâce aux tests automatisés

Plus globalement, cette approche permet de mettre en œuvre les principes DevOps en assurant une intégration continue et un déploiement rapide et fiable.

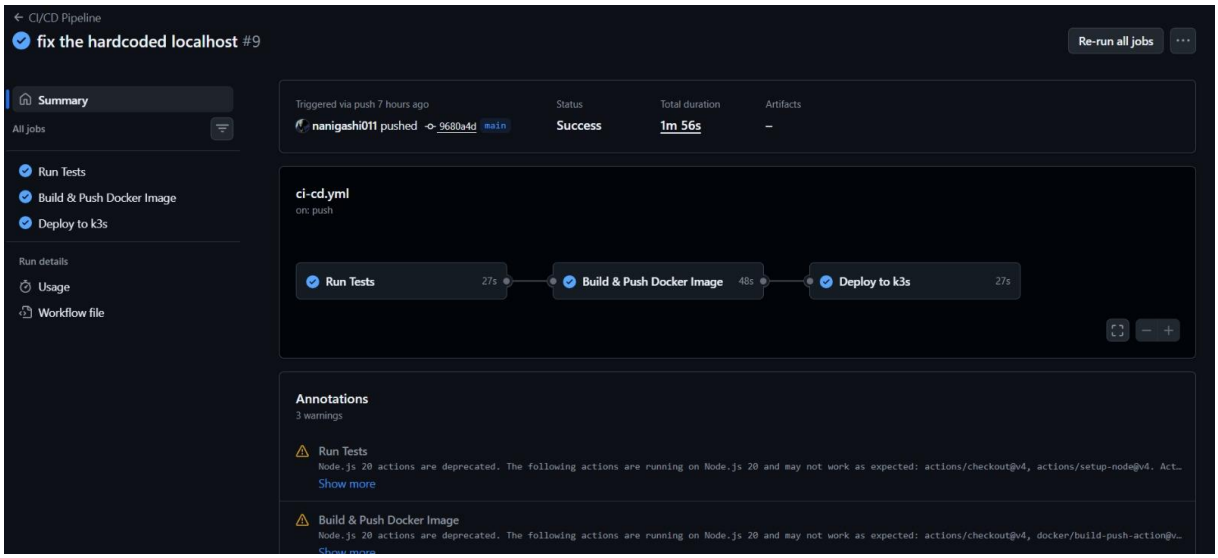

Figure 16 : Exécution réussie du pipeline CI/CD avec GitHub Actions

Cette figure illustre l'exécution du pipeline CI/CD automatisé. Les différentes étapes, incluant les tests, la construction de l'image Docker et le déploiement sur Kubernetes, ont été exécutées avec succès, confirmant le bon fonctionnement du processus d'intégration et de déploiement continus.

9. Tests automatisés

Dans le cadre de ce projet, des **tests automatisés** ont été mis en place afin de garantir la fiabilité et la qualité de l'application avant son déploiement.

Ces tests ont été développés à l'aide de l'outil **Jest**, largement utilisé dans l'écosystème Node.js pour la réalisation de tests unitaires et d'intégration. Ils permettent de vérifier le bon fonctionnement des différentes fonctionnalités de l'API de manière automatique.

L'objectif principal de ces tests est de détecter les erreurs le plus tôt possible dans le cycle de développement, afin d'éviter le déploiement d'une version instable de l'application. Ils sont ainsi intégrés directement dans le pipeline CI/CD, où ils sont exécutés automatiquement à chaque modification du code.

Plusieurs scénarios de test ont été implémentés, notamment :

- la vérification de l'accès aux routes principales de l'API ;
- le test de récupération des données ;
- le test de création d'un nouvel élément ;
- la vérification de l'état du serveur via l'endpoint `/api/health`.

Ces tests permettent de s'assurer que l'API répond correctement aux requêtes et que les fonctionnalités essentielles sont opérationnelles.

L'intégration des tests automatisés dans le pipeline CI/CD constitue un élément clé de l'approche DevOps, puisqu'elle contribue à améliorer la qualité du code, à réduire les risques d'erreurs en production et à renforcer la confiance dans les déploiements.

10. Monitoring

Dans le cadre de ce projet, un système de **monitoring** a été mis en place afin d'assurer la supervision de l'infrastructure et de l'application en temps réel. Cette étape est essentielle dans une démarche DevOps, car elle permet de garantir la stabilité du système et de réagir rapidement en cas de problème.

Le monitoring repose sur l'utilisation de deux outils complémentaires : **Prometheus** et **Grafana**.

Prometheus est utilisé comme système de collecte de métriques. Il interroge régulièrement les différents composants du système (notamment l'application déployée sur Kubernetes) afin de récupérer des informations telles que l'état du service, les performances ou encore la disponibilité.

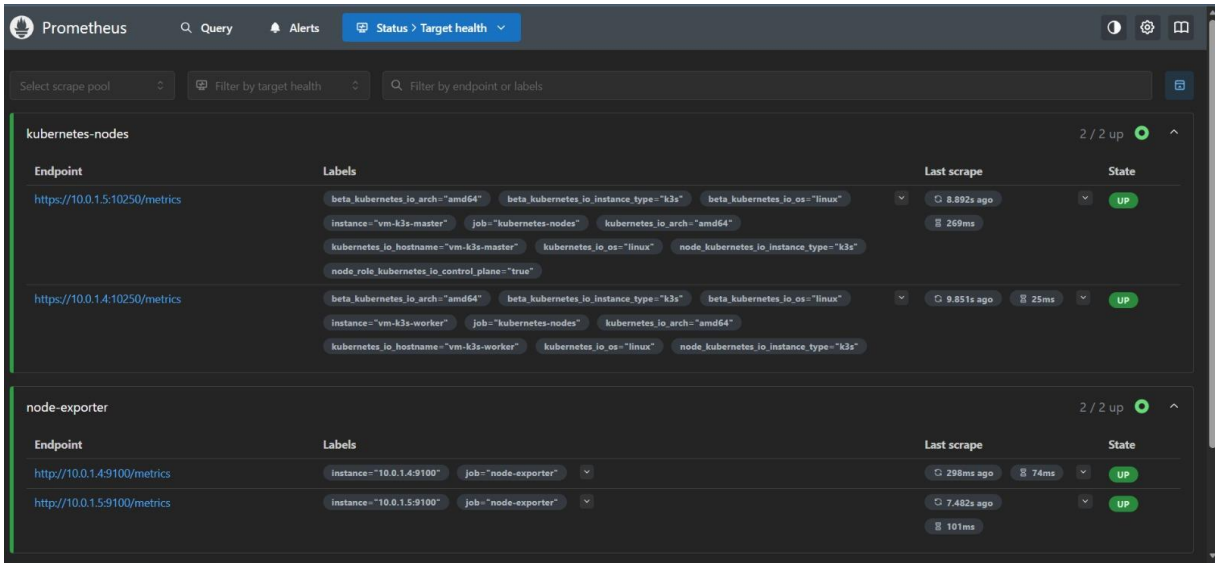

Figure 17 : État des cibles surveillées dans Prometheus

Cette figure présente l'interface Prometheus affichant l'état des différentes cibles surveillées. On observe que les nœuds du cluster Kubernetes ainsi que les instances Node Exporter sont en état "UP", ce qui signifie que la collecte des métriques est opérationnelle. Cela confirme que Prometheus est correctement configuré et qu'il est capable de superviser l'ensemble des composants de l'infrastructure.

Grafana, quant à lui, permet de visualiser ces données sous forme de tableaux de bord interactifs. Il offre une interface claire et dynamique permettant d'analyser les performances du système en temps réel.

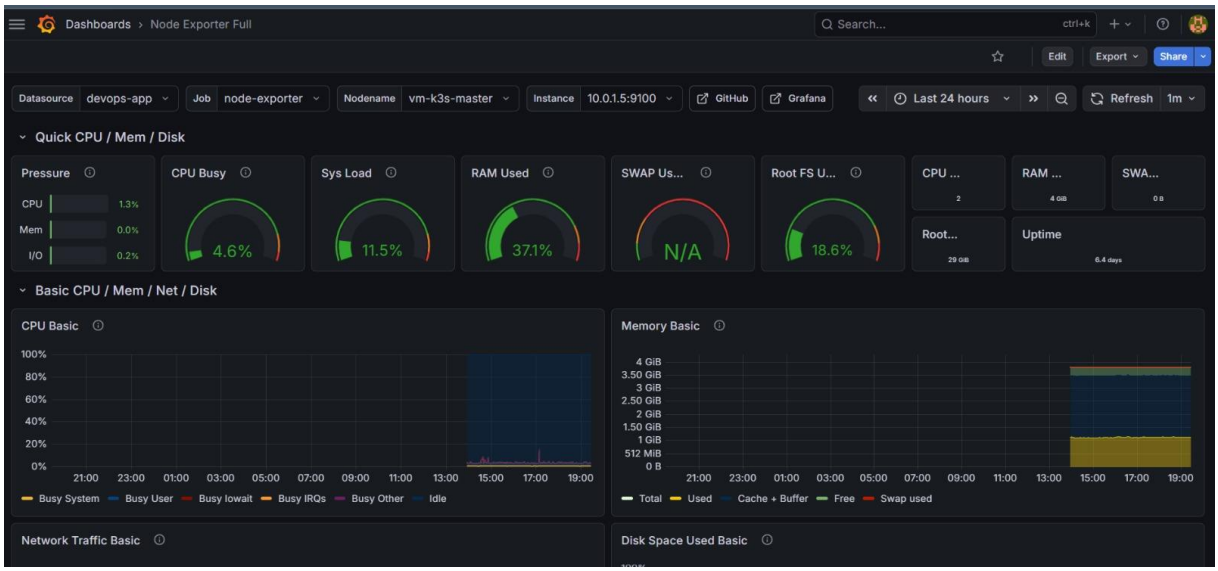

Figure 18 : Tableau de bord Grafana illustrant les métriques du système

Cette figure illustre un tableau de bord Grafana affichant les métriques du système en temps réel, notamment l'utilisation du CPU, de la mémoire, du disque ainsi que la charge du système. Ces indicateurs permettent de suivre les performances de l'infrastructure et de détecter d'éventuelles anomalies, contribuant ainsi à une meilleure supervision du système.

L'objectif principal de ce système de monitoring est double. D'une part, il permet de **suivre les performances** de l'application et de l'infrastructure. D'autre part, il facilite la **détection rapide des anomalies**, telles qu'une indisponibilité du service ou une dégradation des performances.

Le fonctionnement global repose sur un flux simple : Prometheus collecte les métriques à intervalles réguliers, puis Grafana les exploite pour afficher des indicateurs clés sous forme de graphiques et de tableaux de bord.

Cette approche permet d'améliorer la visibilité sur le système, d'anticiper les problèmes potentiels et de garantir un meilleur niveau de fiabilité en production.

11. Résultats

La mise en œuvre de ce projet a permis de déployer avec succès une **infrastructure DevOps complète**, couvrant l'ensemble du cycle de vie de l'application, depuis son développement jusqu'à sa mise en production et sa supervision.

Les différents composants intégrés (Docker, Terraform, Ansible, Kubernetes, CI/CD et monitoring) ont été correctement configurés et interconnectés, permettant d'obtenir un système cohérent et fonctionnel.

Plusieurs résultats concrets peuvent être mis en évidence.

Tout d'abord, l'application développée est pleinement fonctionnelle et accessible après son déploiement. Elle répond correctement aux différentes requêtes API, ce qui valide le bon fonctionnement de la partie applicative.

Ensuite, le pipeline **CI/CD** mis en place permet un **déploiement entièrement automatisé**. À chaque modification du code source, les tests sont exécutés, l'image Docker est construite, puis l'application est redéployée automatiquement sur le cluster Kubernetes, sans intervention manuelle.

Par ailleurs, le cluster Kubernetes est **opérationnel et stable**. Il assure la gestion des pods, la continuité du service ainsi que le redémarrage automatique en cas de défaillance, garantissant ainsi la résilience de l'application.

Enfin, un système de **monitoring actif** a été déployé. Grâce à Prometheus et Grafana, il est possible de suivre en temps réel les performances du système et de détecter d'éventuelles anomalies.

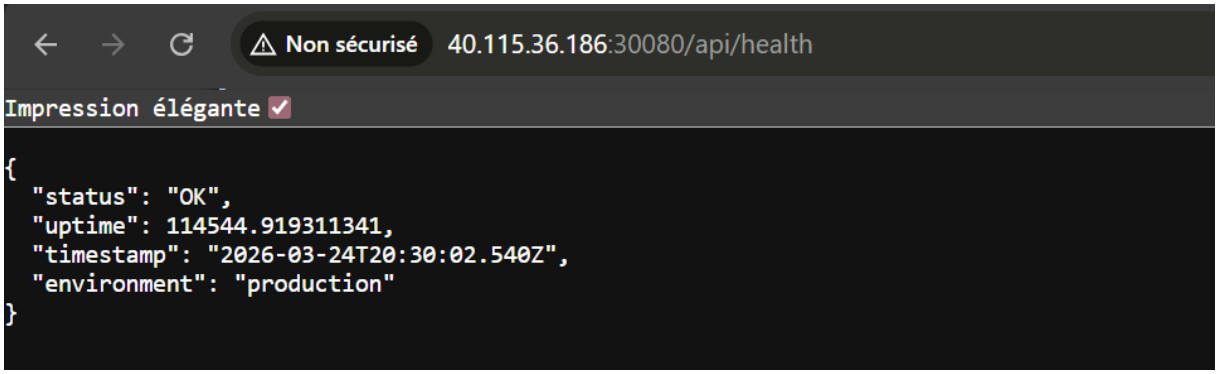

Figure 19 : Vérification de l'état de l'application via endpoint /api/health

Cette figure montre la réponse de l'endpoint /api/health accessible via l'URL publique.
Le statut "OK" confirme que l'application est correctement déployée et fonctionnelle en production.

Dans l'ensemble, ces résultats démontrent la réussite du projet ainsi que la mise en œuvre effective des principes DevOps dans un environnement proche des conditions réelles.

12. Conclusion

Ce projet a permis de mettre en pratique les principes fondamentaux du **DevOps** à travers la conception et la mise en œuvre d'une infrastructure complète, intégrant les différentes étapes du cycle de vie d'une application.

À travers ce travail, nous avons pu automatiser l'ensemble du processus, depuis le développement de l'application jusqu'à son déploiement sur un cluster Kubernetes, en passant par la conteneurisation, l'intégration continue et la configuration de l'infrastructure. Cette approche a permis de mieux comprendre l'importance de l'automatisation, de la reproductibilité et de la collaboration entre les différentes composantes du système.

L'intégration des outils tels que Docker, Terraform, Ansible, Kubernetes et GitHub Actions a permis de construire une chaîne DevOps cohérente et fonctionnelle, proche des pratiques adoptées dans les environnements professionnels. De plus, la mise en place d'un système de monitoring avec Prometheus et Grafana a renforcé la visibilité sur le système et la capacité à détecter les anomalies.

Au-delà des aspects techniques, ce projet a également permis de mettre en évidence les enjeux liés à la gestion d'infrastructures distribuées, à la fiabilité des déploiements et à la qualité du code dans un contexte d'intégration continue.

Malgré les résultats obtenus, certaines améliorations peuvent être envisagées afin d'enrichir davantage le système. Il serait notamment possible d'intégrer des mécanismes de **scaling automatique** afin d'adapter les ressources en fonction de la charge, de renforcer la **sécurité du cluster Kubernetes** (gestion des accès, secrets, certificats) et d'ajouter une **base de données persistante** pour rendre l'application plus proche d'un cas d'usage réel.

En conclusion, ce projet constitue une expérience concrète et formatrice, illustrant l'apport du DevOps dans la modernisation des processus de développement et de déploiement des applications.